

QuickBite: Presentation Script

This script is for a **7-slide** deck and runs about **6½ minutes**. Lines in *italics* are what to do or point at. Everything else is what to say. The same text is in each slide's speaker notes.

#	Slide	Time	Running total
1	Title	0:30	0:30
2	The problem	0:45	1:15
3	ER diagram (you explain)	1:30	2:45
4	Normalization	1:00	3:45
5	Key queries	1:00	4:45
6	Indexing	0:50	5:35
7	Conclusion	0:45	6:20

Before you present: check that the timings on slide 6 match your latest `outputs/05_index_demo_output.txt`. If you ran `./run_all.sh` again, the milliseconds will have changed.

Slide 1: Title (0:30)

Good morning everyone. I'm **Aditya Sunil Chouksey**, and my case study is number 83: **QuickBite, a food delivery order and restaurant management system**.

QuickBite is an aggregator, like Swiggy or Zomato. It lists restaurant menus and processes customer orders.

In the next few minutes I'll cover the problem, the ER diagram, how I

normalized the schema, the key queries, the index I built, and how the database protects its own data. Everything I show was run on a real PostgreSQL 18 database.

Slide 2: The problem (0:45)

QuickBite needs two answers from its data: **how much revenue each restaurant earns**, and **which dish is each restaurant's most popular**, so it can be promoted.

From the brief I took four goals. (*Point to each card.*)

1. **Revenue per restaurant.** That's a GROUP BY with SUM.
2. **Most popular dish.** That uses a JOIN, COUNT and a ranking subquery.
3. **Fast search.** Browsing by cuisine and area is the most common search, so I added a composite index.
4. **Valid orders only.** Every order must belong to a real customer, restaurant and delivery agent.

I'll show how each of these is met.

Slide 3: ER diagram (1:30), your explanation

Use this as a guide, not word for word. **Start at the centre, then work outwards.**

1. (*Point to FOOD_ORDER in the middle.*) "The centre of the design is **FOOD_ORDER**. It has the fields the brief asks for: order ID, customer ID, restaurant ID, delivery agent ID and order time."
2. (*Trace the line to RESTAURANT, top left.*) "One **restaurant** receives many orders, which is one-to-many. The two bars on the restaurant end mean *exactly one*. The circle and crow's foot on

the order end mean *zero or many*: a new restaurant like Malvani Tadka can have no orders yet."

3. (*Right side: CUSTOMER and DELIVERY_AGENT.*) "In the same way, one **customer** places many orders, and one **delivery agent** delivers many orders. Both are one-to-many, and both foreign keys are NOT NULL. Every order must have exactly one customer and one agent."
4. (*Down from RESTAURANT to MENU_ITEM.*) "One restaurant **offers** many menu items. This link is how I removed the redundancy, which I'll explain on the next slide."
5. (*Point to ORDER_ITEM, bottom centre.*) "Orders and dishes are **many-to-many**: one order has many dishes, and one dish appears in many orders. A relational table can't store that directly, so **ORDER_ITEM** sits in between. Its primary key is the pair order ID plus menu item ID. It also stores **quantity** and **unit price**, the price at the time of the order."
6. (*Point to PAYMENT, bottom right.*) "**PAYMENT** is one-to-one with the order. That comes from a foreign key that is also **UNIQUE**, so an order can never have two payments."
7. (*Point to the tags or the legend.*) "The tags mark the keys: **PK** for primary key, **FK** for foreign key, **UQ** for unique, and **IX** for the two columns in the search index."

Likely follow-up questions:

- "Why did you add ORDER_ITEM and PAYMENT?" The brief asks for 1:1, 1:M and M:N relationships. M:N needs a junction table, and that table is also the "order-lines" the required query joins on. Payment is a natural 1:1.
 - "What do the symbols mean?" Two bars mean exactly one. A crow's foot with a bar means one or many. A crow's foot with a circle means zero or many.
-
-

Slide 4: Normalization (1:00)

The brief points out a redundancy: the restaurant's **cuisine and address were repeated on every menu-item row**. (*Point left.*) You can see Punjab Da Dhaba, North Indian, Kharghar, written three times.

The key of that sheet is restaurant plus dish, but cuisine and address depend on **the restaurant alone**, which is only part of the key. That's a **partial dependency**, so the sheet breaks **second normal form**. It causes anomalies: if the restaurant moves, every row has to change.

(*Point right.*) The fix: restaurant details are stored **once** in a restaurant table, and menu_item points to it with a foreign key. That gives 2NF.

(*Point to the bottom chips.*) For 3NF, I don't store order totals. A view calculates them, so they can't go out of date. And in every table, everything that determines another column is a candidate key, so the schema is in **BCNF**.

Slide 5: Key queries (1:00)

These are the two queries the brief asks for, with real output.

(*Left card.*) The **most popular dish** for each restaurant:

- I JOIN the order lines with the menu items, GROUP BY restaurant and dish, and COUNT the orders containing each dish.
- A **ranking subquery** in HAVING keeps only the dish whose count equals that restaurant's highest. It's **correlated**: it runs once per restaurant.
- Butter Chicken wins at Punjab Da Dhaba with 6 orders.
- (*Point to Dragon Wok.*) Two dishes **tie** at 3, and the query keeps both. I put that tie in the data on purpose, to prove ties are handled.

(*Right card.*) **Revenue per restaurant:**

- It's a GROUP BY with SUM over the order totals, delivered orders only.
 - Biryani Darbar earns the most, at ₹5,270.
 - Because I used a **LEFT JOIN**, Malvani Tadka, with no orders, still appears at zero instead of disappearing.
-

Slide 6: Indexing (0:50)

The advanced concept is **indexing**. Browsing by cuisine and area is the app's most common search, so I created a composite B-tree index on **cuisine and location**.

To prove it works, I loaded **200,000 test restaurants** inside a transaction and ran "North Indian in Kharghar" with EXPLAIN ANALYZE.

- *(Left number.)* Without the index: a **sequential scan** that reads every row, about **13 milliseconds**.
- *(Right number.)* With the index: a **bitmap index scan** that reads only the **835 matching rows**, about **1 millisecond**. That's roughly **13 times faster**.

Then I rolled the transaction back, so the test rows never touched the real data.

Column order matters. Cuisine comes first, so the same index also speeds up **cuisine-only** searches. That's the leftmost-prefix rule.

Slide 7: Conclusion (0:45)

To conclude: **all four outcomes from the brief are met**, and each one was verified on a live database.

(Point to "Delivered".)

- A schema of **7 tables in BCNF**, with a full ER diagram.
- The two required queries: **most popular dish** and **revenue per restaurant**.
- The composite index, which made the cuisine-and-area search **about 13 times faster** on 200,000 rows.
- Integrity: I tried **nine invalid writes**, such as an order for a customer that doesn't exist, a biryani on a pizza order, or a second payment for one order. **All nine were rejected** by the database itself, through foreign keys, NOT NULL, the trigger, RESTRICT, UNIQUE and CHECK.

(Point to "Next steps".) If QuickBite grew, I would:

- partition the orders table by month
- turn the order-total view into a **materialized view** for faster revenue reports
- move cuisine and area into **lookup tables**, so spellings stay consistent
- add real distance-based search with **PostGIS**

Everything is in my GitHub repository. **Thank you. I'm happy to take questions.**

Delivery tips

- **Rehearse slide 3 the most.** It's your longest slide and you're explaining it yourself. Practise tracing the lines in the same order every time.
- Point at the slide while you talk. Don't read it aloud.
- If you're running long, cut the leftmost-prefix sentence on slide 6 and the "Next steps" part of slide 7.
- For deeper questions about the trigger, the view, why unit_price is stored, or when the index isn't used, use the answers in [VIVA NOTES.md](#).